

Chapitre 3

Configuring Application Layer Services

1

Outline

- Hosting HTTP Content via Apache
- Setting up an internal FTP server

2

Hosting HTTP Content via Apache

3

Apache

- Adding a Web server (Apache) in the same LAN as DNS but in its own namespace is the natural way, assuming routing (**1_NetConf**), NAT (**2_add_nat_internet**) and DNS with name resolution working (**5_enabling_dns**, **5_enabling_resolo**).
- This requires:
 - Create a WEB namespace
 - Place it in another LAN (10.0.101.0/24)
 - Install & run Apache inside the WEB namespace
 - Access the website from Cl and Cr
 - Resolve it using DNS name (web.lab.local)

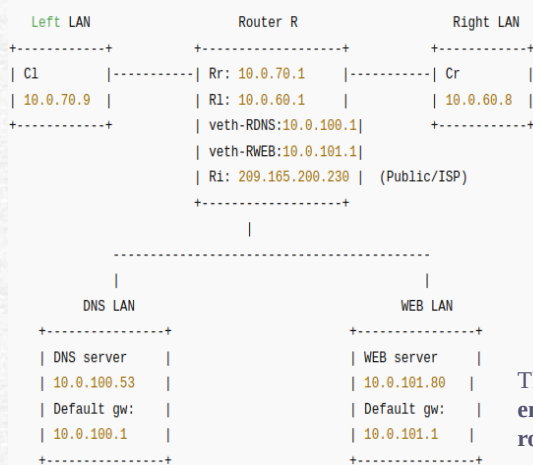
Device	Role	IP	Notes
R	Router	10.0.100.2	Interface toward DNS/Web LAN
DNS	DNS server	10.0.100.53	Default route via 10.0.100.1
WEB	Web server	10.0.100.80	Default route via 10.0.101.1
Cl	Client	10.0.70.9	Default route via 10.0.70.1 (Router internal)
Cr	Client	10.0.60.8	Default route via 10.0.60.1 (Router internal)

4

Apache

CI → Router R → DNS → Router R → CI
 ip netns exec CI dig web.lab.local
 web.lab.local → 10.0.100.80

CI → Router R → WEB (Apache)
 ip netns exec CI curl http://web.lab.local



- Creates a test HTML page
- Starts the server in the WEB namespace
- Tests connectivity **from WEB, CI, and Cr**
- Prints the actual HTML content fetched from CI and Cr

This setup simulates a **real enterprise web server behind a router.**

5

Apache

- **Create the WEB namespace**
 - sudo ip netns add WEB
 - sudo ip netns exec WEB ip link set lo up
- **Connect WEB to Router R (new LAN)**
 - sudo ip link add veth-WEB type veth peer name veth-RWEB
 - sudo ip link set veth-WEB netns WEB
 - sudo ip link set veth-RWEB netns R
- **Assign IP addresses**
 - sudo ip netns exec WEB ip addr add 10.0.101.80/24 dev veth-WEB
 - sudo ip netns exec WEB ip link set veth-WEB up
 - sudo ip netns exec WEB ip route add default via 10.0.101.1
 - sudo ip netns exec R ip addr add 10.0.101.1/24 dev veth-RWEB
 - sudo ip netns exec R ip link set veth-RWEB

- **Test connectivity**

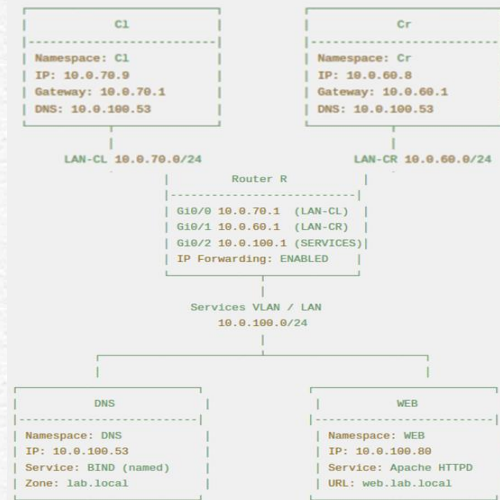
```
sudo ip netns exec $WEB_NS ping -c 2 10.0.101.1
sudo ip netns exec CI ping -c 2 10.0.101.80
sudo ip netns exec Cr ping -c 2 10.0.101.80
sudo ip netns exec $WEB_NS ping -c 2 10.0.100.53
```

```
... 10.0.101.80 ping statistics ...
2 packets transmitted, 2 received, 0% packet loss, time 1010ms
rtt min/avg/max/mdev = 0.033/0.033/0.034/0.000 ms
[OK] Cr -> WEB
Ping from WEB -> DNS:
PING 10.0.100.53 (10.0.100.53) 56(84) bytes of data:
64 bytes from 10.0.100.53: icmp_seq=1 ttl=63 time=0.033 ms
64 bytes from 10.0.100.53: icmp_seq=2 ttl=63 time=0.041 ms
... 10.0.100.53 ping statistics ...
2 packets transmitted, 2 received, 0% packet loss, time 1017ms
rtt min/avg/max/mdev = 0.033/0.037/0.041/0.004 ms
[OK] WEB -> DNS
=== WEB namespace setup complete ===
WEB reachable at 10.0.101.80 from CI, Cr, and can reach DNS at 10.0.100.53
mahouya@ubuntu: ~$
```

6

Apache

- **Exercise:** Use a bridge to have DNS and WEB in the same subnet, as shown in this figure.



7

Apache

- The **objective** is to (**6_enabling_web**):
 - Run a web server inside the WEB network namespace
 - Serve a real HTML page
 - Access this page from C1 and Cr
 - Visually confirm that routing works by seeing the page content
- **Create the web page directory**, will contain the web page served to clients.
 - `sudo ip netns exec WEB mkdir -p /tmp/webpage.`
- **Create a test HTML page**
 - `sudo ip netns exec WEB bash -c "cat > /tmp/webpage/index.html <<EOF`

```
<!DOCTYPE html>
<html> <head> <title>WEB Test</title></head><body>
<h1>WEB Server is Running!</h1>
<p>This page is served from namespace WEB (IP: 10.0.101.80)</p>
</body></html>
EOF"
```
- **Start the Python HTTP server**
 - `sudo ip netns exec WEB bash -c "cd /tmp/webpage && python3 -m http.server 80 --bind 0.0.0.0 &"`

8

Apache

- **Verify that port 80 is listening (6_enabling_apache).**
 - `sudo ip netns exec WEB ss -ltnp | grep :80`
- **Test the web server from inside WEB**
 - `sudo ip netns exec WEB curl -I http://localhost:80`
- **Access the web server from Cl**
 - `sudo ip netns exec Cl curl http://10.0.101.80:80`

```

=== Content received by Cr ===
<!DOCTYPE html>
<html>
<head>
<title>WEB Namespace Test</title>
</head>
<body>
<h1>WEB Namespace Server is Running!</h1>
<p>This page is served from namespace WEB (IP: 10.0.101.80)</p>
</body>
</html>
=== Step 9: Summary ===
Python HTTP server with test page is running in WEB namespace
WEB server IP: 10.0.101.80, listening on port 80
Clients Cl and Cr can access the page via http://10.0.101.80:80
bakhouya@ubuntu:~/Documents/Cours/6-Web$

```

9

Apache

- The web server is internal, no external access.
 - The web server is running on a private IP address
 - It exists inside the lab network, in a Linux network namespace
 - It is not directly visible from outside the network (Internet / ISP)
 - Private IP addresses cannot be reached directly from external networks.
- Exposing the HTTPS web server to an external network
 - Destination Network Address Translation (DNAT) on the Gateway.
 - Demonstrates how inbound access to private services is enabled in real-world networks while preserving internal IP addressing and security boundaries.

See, Lab 5 for exposing the HTTP web server to an external network using DNAT including HTTPS.

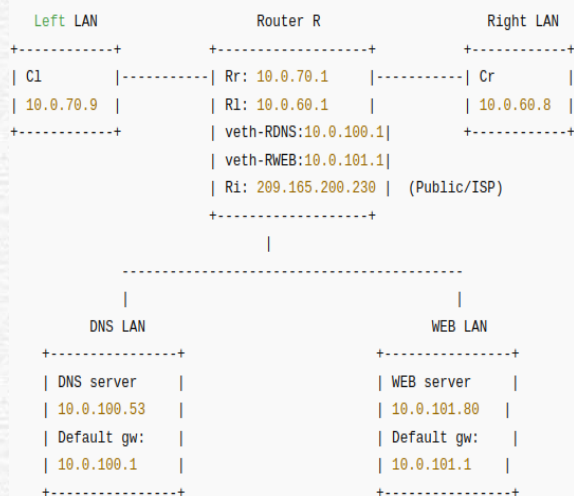
10

Setting up an internal FTP server

11

FTP

- Install an FTP server on the same WEB server that already hosts HTTP
- Allow internal clients only (Cl and Cr) to access FTP
- Share files using FTP
- Understand how multiple (put and get) services run on the same server (**7_enabling_ftp**)



12

FTP

- Assume vsftpd is already installed on the host, run once.
 - sudo apt update
 - sudo apt install -y vsftpd
- Stop running FTP and backup configuration (Good administrative practice).
 - sudo pkill vsftpd 2>/dev/null || true
 - sudo cp /etc/vsftpd.conf /etc/vsftpd.conf.bak 2>/dev/null || true
- Create New FTP Configuration
 - sudo bash -c "cat > /etc/vsftpd.conf <<EOF


```
listen=YES                # IPv4 only
listen_ipv6=NO
write_enable=YES          # File upload
anon_upload_enable=YES    # Directory creation
anon_mkdir_write_enable=YES # Write permissions
anon_root=/srv/ftp        # The root directory for FTP users.
no_anon_password=YES      # Anonymous users don't need to enter a
password
pasv_enable=YES           # Firewall must allow TCP 21, TCP 30000–
30010
pasv_min_port=30000       # Restricts data ports to a small known range
pasv_max_port=30010       # Easier to filter with iptables
xferlog_enable=YES        # Transfer logs
chroot_local_user=YES      # Users cannot leave /srv/ftp.
```

13

FTP

- Create Directory Structure
 - sudo rm -rf /srv/ftp # Deletes any previous FTP directory.
 - sudo mkdir -p /srv/ftp/upload #
 - sudo chmod 755 /srv/ftp # must NOT be writable, otherwise vsftpd will not start.
 - sudo chmod 777 /srv/ftp/upload # Upload folder is writable.
- Add Test Files
 - echo 'Internal FTP Server (WEB 10.0.101.80)' > /srv/ftp/README.txt
 - date > /srv/ftp/server_time.txt
- Start FTP and verify Service
 - sudo ip netns exec WEB vsftpd /etc/vsftpd.conf &
 - sudo ip netns exec WEB ss -lntp | grep ':21'

```
bakhouya@ubuntu:~/Documents/Cours/7.FTP$ sudo ip netns exec WEB ls /srv/ftp/uplo
ad
bakhouya@ubuntu:~/Documents/Cours/7.FTP$ sudo ip netns exec WEB ls /srv/ftp/uplo
ad
test
```

14

FTP

- Put and get files

```
bakhouya@ubuntu:~/Documents/Cours/7.FTP$ sudo ip netns exec Cl ftp 10.0.101.80
Connected to 10.0.101.80.
220 (vsFTPD 3.0.5)
Name (10.0.101.80:bakhouya): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
-rw-r--r-- 1 0 0 38 Feb 11 16:07 README.txt
-rw-r--r-- 1 0 0 32 Feb 11 16:07 server_time.txt
drwxrwxrwx 2 0 0 4096 Feb 11 16:07 upload
226 Directory send OK.
ftp> cd upload
250 Directory successfully changed.
ftp> put test
local: test remote: test
200 PORT command successful. Consider using PASV.
150 Ok to send data.
226 Transfer complete.
ftp>
```

15

References

- Paul Cobbaut, Linux Networking, <http://linux-training.be/>
- Linux Network Administrator's Guide, 2nd Edition, by Olaf Kirch & Terry Dawson; published by O'Reilly & Associates, Inc. 503 pages; <https://www.linuxdoc.org/>
- Jay LaCroix, Mastering Linux Network Administration, Packt Publishing Ltd. ISBN 978-1-78439-959-7, www.packtpub.com
- Remo Suppi Boldrito, Network administration, <https://openaccess.uoc.edu/>

16